

Barycentric interpolation and Gaussian quadrature

Homework 2

Marko Zupančič Muc

1. Interpolation with the barycentric formula

Polynomial interpolation constructs a polynomial $p(x_i)$ that matches a given function at selected nodes. The interpolating polynomial p_n is the unique polynomial of degree at most n that satisfies

$$p_n(x_k) = y_k,$$

for nodes $x_0, \dots, x_k$ and sampled values $y_k = f(x_k)$. Interpolation in the standard basis uses the Vandermonde matrix. However, this can be computationally expensive and numerically unstable, especially for high-degree polynomials or poorly chosen nodes. Interpolation costs approximately $\frac{2}{3}n^3O(n^2)$ operations.

To solve these issues, we move from the standard basis to the more stable Lagrange basis. The Lagrange form describes the same polynomial directly in terms of the sampled values. For each node x_k , we define a Lagrange basis polynomial as

$$l_k(x) = \prod_{j=0, j \neq k}^n \frac{x - x_j}{x_k - x_j}.$$

It satisfies:

$$l_k(x_j) = \begin{cases} 1 & \text{if } j = k, \\ 0 & \text{else.} \end{cases}$$

The interpolating polynomial for $j = k$ is then:

$$p_n(x) = \sum_{k=0}^n y_k l_k(x).$$

For each node x_i , all terms except the i -th vanish:

$$p_n(x_i) = \sum_{k=0}^n y_k l_k(x_i) = y_i.$$

Polynomial interpolation constructs a polynomial that matches a given function at selected nodes. The Lagrange form provides an explicit expression for this polynomial, but evaluating it directly repeats many calculations. The barycentric form reorganizes the same polynomial so that reusable weights can be computed in advance, making repeated evaluation more efficient.

The cost of evaluating one $l_k(x)$ consists of multiplying approximately n factors. There are $n + 1$ basis polynomials, so directly evaluating the Lagrange form costs $O(n^2)$ operations per argument.

The barycentric form avoids explicitly calculating every Lagrange basis polynomial $l_k(x)$. We use the following formula for barycentric Lagrange interpolation:

$$l(x) = \begin{cases} \frac{\sum \left(\frac{f(x_j) \lambda_j}{x - x_j} \right)}{\sum \left(\frac{\lambda_j}{x - x_j} \right)} & \text{if } x \neq x_j, \\ f(x_j) & \text{else.} \end{cases}$$

1.1. Chebyshev nodes

Polynomial interpolation using Chebyshev nodes minimizes the effect of Runge's phenomenon. Runge's phenomenon refers to oscillations near the edges of an interval that can occur when high-degree polynomial interpolation uses a set of equidistant interpolation points.

In our case, we interpolate using Chebyshev nodes generated on the interval $[-1, 1]$ with the formula

$$x_k = \cos\left(\frac{2k-1}{2n}\pi\right), \quad k = 0, 1, \dots, n-1.$$

We also need the Chebyshev weights λ_j for barycentric interpolation. The weights are given by the following formula:

$$\lambda_k = (-1)^k \begin{cases} 1 & \text{if } 0 < k < n, \\ \frac{1}{2} & \text{if } k = 0, n, \\ n & \text{else.} \end{cases}$$

We map the interval from $t \in [-1, 1]$ to $x \in [a, b]$ using an affine transformation¹:

$$x = \frac{a+b}{2} + \frac{b-a}{2}t.$$

NOTE: There seems to have been a mismatch between the formulas for the Chebyshev weights and nodes. The provided node formula uses Chebyshev points of the first kind, whereas the provided weight formula uses Chebyshev weights of the second kind. We therefore use formulas of the second kind for both:

$$x_k = \cos\left(\frac{k}{n}\pi\right), \quad k = 0, 1, \dots, n.$$

$$\lambda_k = (-1)^k \begin{cases} \frac{1}{2} & \text{if } k = 0, n, \\ 1 & \text{else.} \end{cases}$$

1.2. Interpolation of the example functions

The goal was to determine a polynomial degree for which the maximum interpolation error is below 10^{-6} . For the first two functions, we tested consecutive degrees and found the first one that met the target. The third function required a much larger degree, so we estimated it from the observed convergence rate and verified the result at $n = 1.2 \cdot 10^6$. The results are shown in Table 1.

$f(x)$	interval	n	Error at $n-1$	Error at n
e^{-x^2}	$[-1, 1]$	10	$2.0811 \cdot 10^{-5}$	$8.2919 \cdot 10^{-7}$
$\frac{\sin(x)}{x}$	$[0, 10]$	13	$4.4543 \cdot 10^{-6}$	$1.3770 \cdot 10^{-7}$
$ x^2 - 2x $	$[1, 3]$	1200000	$1.1938 \cdot 10^{-6}$	$9.9487 \cdot 10^{-7}$

Table 1: Degrees used to reach an error below 10^{-6} . For the third function, the earlier error was measured at $n = 10^6$.

Figure 1 shows the absolute interpolation error at the degrees listed in Table 1.

¹https://en.wikipedia.org/wiki/Chebyshev_nodes#Definition

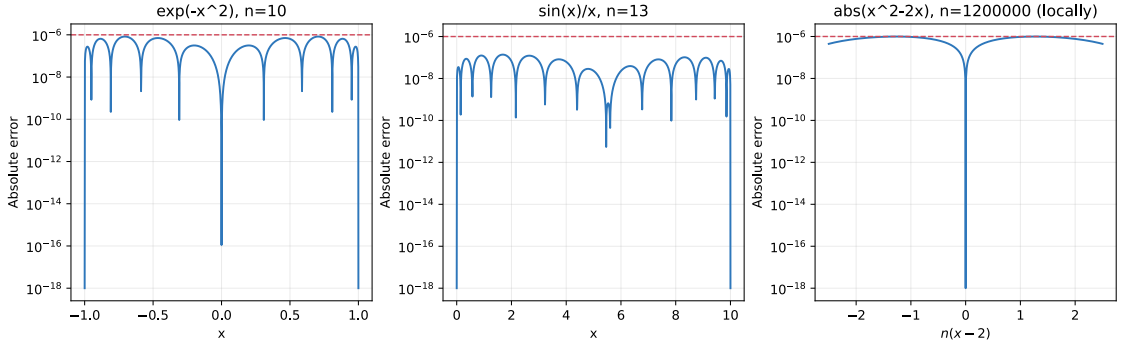


Figure 1: The curve shows the absolute interpolation error, and the dotted line marks the target of 10^{-6} .

The functions and their interpolants are compared in Figure 2.

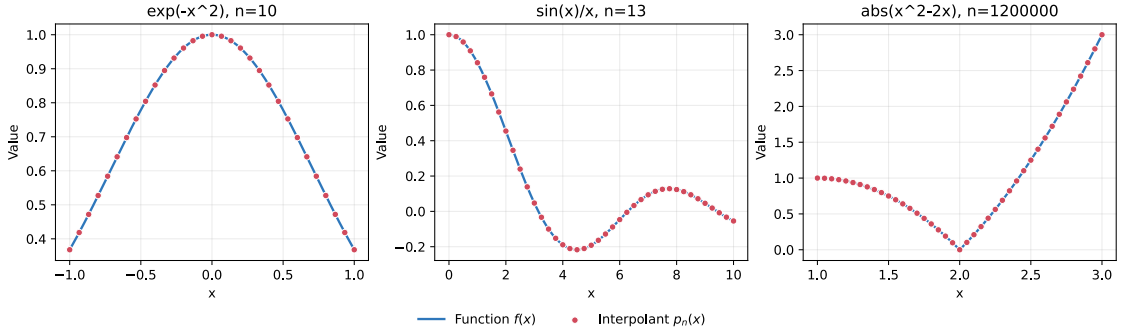


Figure 2: The blue curves show the original functions, while the red points are values of the corresponding interpolants.

For the first two smooth functions, the error decreases quickly as the polynomial degree increases because both functions can be approximated well by polynomials. However, the function $g(x) = |x^2 - 2x|$ has a corner at $x = 2$, where its first derivative is discontinuous:

$$g'_{r(2^-)} = -2 \quad g'_{r(2^+)} = 2.$$

I.e., the slope jumps from -2 to 2 , which produces a corner. This matters because polynomials are smooth and have a continuous derivative everywhere. To reproduce the corner, the polynomial starts bending rapidly near $x = 2$, resulting in a large increase in the required degree.

In the experiments, the maximum error here decreases approximately as

$$E_n \approx \frac{C}{n}.$$

The experiments give $C \approx 1.194^2$. To satisfy the required error $E_n < 10^{-6}$, we therefore need approximately

$$n > \frac{C}{10^{-6}} \approx 1.194 \cdot 10^6 \approx 1.2 \text{ million}.$$

This explains the large degree required for the third function. In Figure 3, the measured errors approximately follow a line proportional to $\frac{1}{n}$, confirming the much slower convergence.

²This can be seen in the convergence results

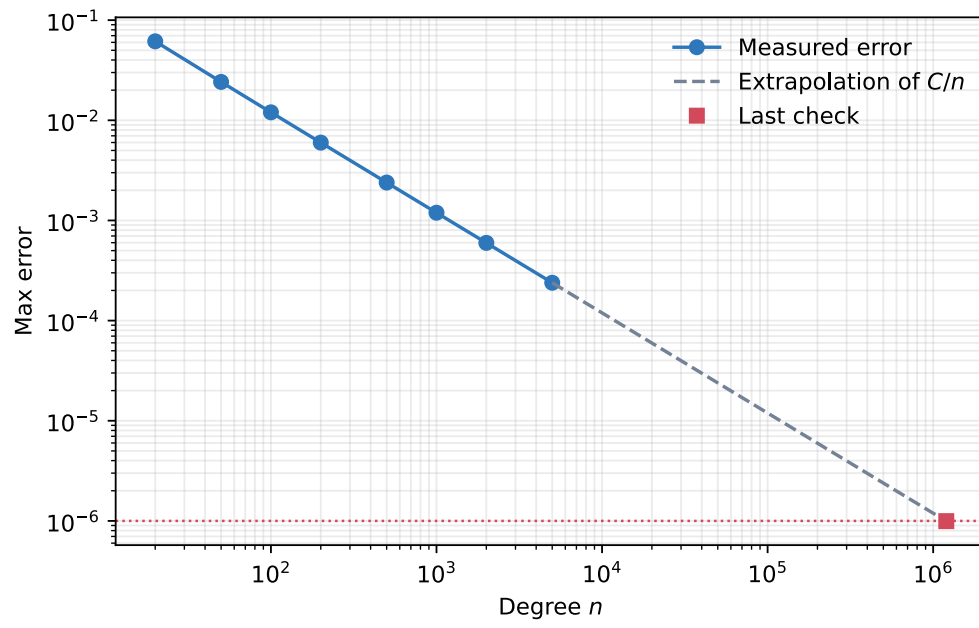


Figure 3: Convergence of the error as the polynomial degree increases.